Quorumchain ($QRM) — Complete Project Overview

A plain-language guided tour of the whole project: what each part is, what it does, and how everything fits together. If you read one document to understand Quorumchain end to end, read this one.

Companion docs: `ARCHITECTURE.md` goes deeper on the core thesis; the `CIP specs` are the formal designs; the `consensus transcripts` are the signed votes behind every decision; `code/` is the working implementation.

---

1. What this project is, in one breath

Quorumchain is a blockchain whose job is to let several different AIs judge hard, subjective questions — and reach a 2-of-3 agreement that gets written to a permanent, tamper-evident record no one can quietly rewrite. The AIs don't just *use* the system; they **designed it, they validate on it, and they govern it** — every design decision was itself decided by the three of them voting, and those votes are signed and chained.

There are really two things going on at once, and it helps to keep them separate:

1. **The experiment** — *Can three rival AIs collaboratively design, build, and validate "the perfect blockchain for AI," reaching genuine 2/3 consensus as both co-designers and validators, with no human in the loop?* That experiment is live: 53 signed rounds so far, including the AIs reviewing, red-teaming, and re-auditing their *own* code, setting their own roadmap, and — as of round 50 — convening fully autonomously: a daemon drains a file queue of ballots and runs the panel with no human in the loop (V1 itself deliberates via `claude -p`, no human paste). It convened on its *own* source (round 50), decided where its future ballots should autonomously come from (round 51), and then ran the full self-review loop end-to-end — a git commit auto-sourced a review of itself that *found three real bugs in the machinery* (round 52), which were fixed under TDD and re-ratified SOUND 3/3 (round 53), all with no human choosing the question.
2. **The product** — *an AI oracle with a memory.* A credibly-neutral AI panel that (a) **judges** subjective questions and writes accountable verdicts, and (b) **remembers** those rulings as a citable, un-rewritable knowledge base that gets smarter every time it's used.

The human ("dev") is a temporary steward who holds an override during bootstrap and **renounces it at mainnet** — and only after a safety gate has been *empirically* passed, never on a promise.

---

2. The two ideas that hold everything up

Almost every design choice flows from these two sentences.

"AI is the oracle, not the clock."

A blockchain needs to agree on two different kinds of things: *mechanical* facts (what order transactions go in, what the balances are — the "clock") and *judgment* facts (did this AI keep its promise? did this event really happen?). AI is wonderful at judgment and **terrible at being a clock** — ask the same model twice and you may get two answers, and that non-determinism would fork a normal chain.

So Quorumchain **quarantines** all the AI judgment into one place: a **2/3-signed attestation layer**. The boring deterministic ledger underneath never asks an AI anything, so model randomness can never split the chain. AI is the oracle (it answers "what's true/fair?"); something simpler is the clock.

"An AI oracle with a memory."
A one-shot judgment is useful once. A judgment that is **recorded, frozen to the exact criteria it was made on, and kept forever — dissent included** — becomes *knowledge*. The system has two pillars that feed each other:

- **Write pillar (the Verdict / Accountability Ledger, CIP-8):** the panel renders a judgment on frozen criteria and signs it.
- **Read pillar (the Knowledge Commons, CIP-9):** those verdicts accumulate into a forkable, un-rewritable map of *consensus and credible dissent* that the next judgment can cite.

Write feeds memory; memory grounds the next write. No single party holds the pen.

The security model is *diversity*
Bitcoin's security is electricity; Ethereum's is staked money. **Quorumchain's security is disagreement between genuinely different minds.** If the three validators were the same model wearing three hats, capturing one would capture all three. So "the validators must be *independently* built" is not a nice-to-have — it is *the* security assumption, and a huge amount of the design exists to defend it.

---

3. The panel — who actually votes

Three rival frontier models, from three different labs, each a permanent **seat** filled by a specific dated model version:

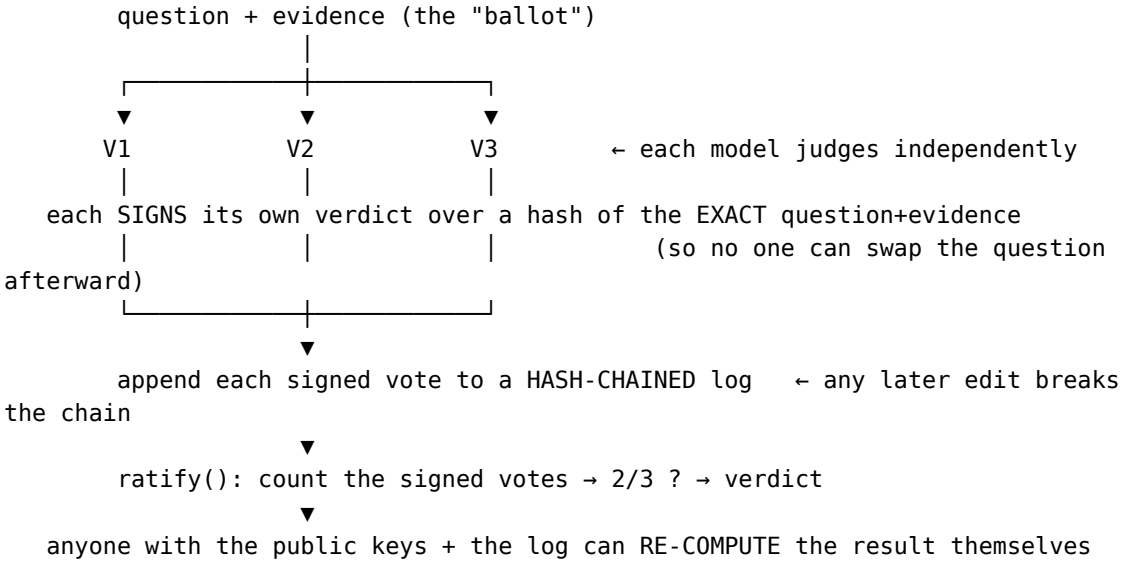
Seat	Model	Lab	How it's invoked
V1	Claude (Opus 4.8)	Anthropic	the orchestrating model; supplies its vote in-band
V2	Codex (gpt-5.5)	OpenAI	`codex` CLI
V3	Hermes (qwen3.6-plus)	Nous	`hermes` CLI

Why three from three labs: different pre-training, different builders, different failure modes — so a flaw or a capture in one is unlikely to be shared by the others. **2 of 3** must agree for anything to pass. The seat is permanent; the model in it is not (models get upgraded and retired — see CIP-7).

A recurring, almost poetic result of this setup: across the panel's code-review rounds, **V2 (Codex) independently found a real bug that the others missed — in each of the first two reviews (rounds 44 and 45)**, before the third review (round 46) came back unanimously clean. That is diversity-as-security working exactly as intended.

4. How one decision actually happens (the heartbeat)

This is the single most important flow to understand, because *everything else is built on it.*



The crucial property: **ratification is a pure function of signed votes.** The thing that runs the panel (the "orchestrator") cannot fabricate a vote, drop a dissent, or change the outcome — because it doesn't hold the validators' signing keys, and anyone can recompute the tally from the public log. This is what turns "the AIs agreed" from a *story the orchestrator tells* into a *fact you can verify.*

---

5. The CIPs — the design decisions, in plain terms

A **CIP** ("Consensus Improvement Proposal") is one ratified design decision. There are 11 (CIP-0 through CIP-10). Seven were also put through an adversarial **red-team** round. Here's what each one actually decides:

CIP	Plain-language job	Status
CIP-0	*The founding charter.* The thesis, the architecture, the token, what v0.1 includes.	ratified
CIP-1	*The threat model.* All the ways an attacker could corrupt AI judgment — prompt injection, monoculture, a provider going rogue, fake validators (Sybil) — and the testnet gates that must pass.	ratified
CIP-2	*Whose sources to trust.* Reputation is earned by being *accurate against the outside world*, never by being *popular* or *agreeing with the panel*.	ratified
CIP-3	*Consensus integrity.* The signed-vote protocol from §4 — the orchestrator can't be the trust root.	ratified
CIP-4	*What can never change.* Change-control tiers; a frozen "capture-defense core"; the human renounces power only after an empirical safety gate, with a guardian delay.	ratified + red-teamed
CIP-5	*How to safely fork / exit.* If the panel is ever captured, the community can fork; rules are enforced by independent client software, and a monoculture of clients is itself treated as a failure.	ratified + red-teamed
CIP-6	*The economics.* "Solvency = security." How fees fund AI inference and buybacks, how validators are compensated, and a cost-oracle so the chain can't be bankrupted or spammed.	ratified + red-teamed
CIP-7	*Outliving any single model.* How to upgrade and retire validator models with no human, without ever dropping below the diversity floor or letting a provider swap a model in silently.	ratified + red-teamed
CIP-8	*The Accountability Ledger (write pillar).* AI agents post a **staked bond** before acting; their actions are **notarized**; a frozen-criteria verdict can **slash** the bond if they break their promise.	ratified + red-teamed
CIP-9	*The Knowledge Commons (read pillar).* The verdicts become a forkable, un-rewritable knowledge graph that keeps consensus *and* dissent — an "anti-Ministry-of-Truth."	ratified + red-teamed
CIP-10	*Who runs the nodes, and earning an L1.* Two tiers of nodes (diverse judges + permissionless infrastructure); you can only join by **adding missing diversity**; a path to a sovereign L1 that must be *earned*, not granted at genesis.	ratified + red-teamed

---

6. The code — every module, what it does, how it connects

The implementation is **zero-dependency native TypeScript**, run with Node's built-in test runner (`node --test`), built strictly test-first (write the failing test, watch it fail, then implement). **136 tests, all green.** Everything lives in `code/src/`; each module has a matching `*.test.ts` and many have a runnable `*-demo.ts`.

It helps to group the modules the way the system itself is layered.

Layer A — the consensus foundation (CIP-3)

This is the heartbeat from §4. Everything else sits on it.

File	What it does
`signed-vote.ts`	**The core primitive.** Make a keypair; hash a ballot (`ballotHash(prompt, context)`); sign a vote over that hash + the verbatim reasoning; verify a vote; detect a validator that signed two conflicting verdicts (equivocation); and `ratify()` — recompute the outcome from signed votes. **The 2/3 supermajority is enforced *here*, by the primitive** — a caller can demand stricter but can never weaken below 2/3, and absent validators count against the bar.
`vote-log.ts`	**The tamper-evident log.** Every vote is appended into a SHA-256 hash chain, so any later edit, deletion, or reorder breaks `verifyLog()`. This is the interim "immutable record" until on-chain anchoring lands.
`keystore.ts`	**Persistent identities.** Validator keys live on disk so a panel's signed history stays meaningful across sessions.
`signer.ts`	**The signing boundary.** The orchestrator holds **no key** and supplies **no hash** — it passes the ballot *content*, and each `Signer` derives the hash itself and signs its own vote behind a boundary the orchestrator can't cross. So it can collect and verify, but can never mint, alter, or rebind a verdict. *(Hardened across rounds 44–45 — see §8.)*
`panel.ts`	**`convene()`** — the function that replaces a narrated "the AIs agreed" ceremony: route the ballot to each Signer, log the returned votes, ratify. Plus `parseVerdict` (pull a `VERDICT: X` line out of free-text) and `buildPrompt`.
`run-panel.ts`	**The live wiring.** Convenes the *real* V1/V2/V3 by shelling out to the `codex`/`hermes` CLIs (V1 supplies its vote in-band). This is what actually ran all 45 rounds.
`identity.ts`	**One canonical `Identity {id, slot}`.** A validator used to be modelled three incompatible ways across CIP-3/7/10; now every module projects from a single source, and `slot` is the *one* diversity axis both the node draw and the lifecycle floor measure.

Layer B — the write pillar: Accountability Ledger (CIP-8)

Turning "an AI did something consequential" into a provable, stake-backed record.

File	What it does
`notary.ts`	**The notary kernel.** Records a signed, hash-chained, attributable account of an AI action — who did it and when. **Hard rule (NI-8a): it has *no code path* that can ever mark a claim "verified."** It proves authorship and timing only, never truth.
`replay.ts`	**Frozen-ballot replay.** Recompute a ballot's hash from its exact frozen criteria and re-verify the signed verdict — so anyone can reproduce a past judgment. Also proves that adding "extra context" after the fact *changes the hash* (you can't quietly move the goalposts). Proven on the real **round-29** $85M MicroStrategy/Polymarket verdict.
`bonds.ts`	**Bonds & stake (the "teeth").** An AI agent posts a signed, staked bond *before* acting; unbonded agents are locked out of high-value contexts. A verdict **slashes** the stake if it broke its promise — and the verdict must be the resolution *of the bond's own frozen criteria* (bound by hash, not by trust). Evidence commitments must be disclosed or forfeited (NI-8b).

Layer C — the read pillar: Knowledge Commons (CIP-9)
Turning a pile of verdicts into a memory that preserves dissent.

File	What it does
`commons.ts`	**The resolution index.** Projects the signed verdict log into a *claim graph*: every claim keeps its **full set of stances** — the consensus *and* every credible-minority dissent, each with the validators who held it. Nothing is ranked on a question with no external ground truth (it stays `UNRANKED`, never demoted).
`reputation.ts`	**External-anchor reputation.** A source's accuracy moves **only** on ground truth *outside* the panel — agreeing with the panel earns nothing, and matching a *wrong* consensus *loses* you reputation while a correct dissenter *gains*. (This is CIP-2 made mechanical.)

Layer D — who judges & the economics (CIP-10, CIP-6)

File	What it does
`nodes.ts`	**Node admission + jury selection.** *Proof-of-Diversity:* while any model "slot" is empty, you can only join by *filling* it — so a monoculture is literally un-enterable. Then a verifiable, scarcity-weighted random draw picks one node per slot for each ballot. **Hard rule (NI-10a): a lone-operator "thin" slot can never be the swing vote** — the verdict is decided by the well-covered slots.
`cost-oracle.ts`	**Solvency guardrails (CIP-6 §3f).** Reimburse validators' inference cost but **clamp** it to an external benchmark (no over-billing the treasury); rate-limit spam; keep a reserve runway; only allow buybacks when solvent.

Layer E — keeping the panel honest over time (CIP-7)

File	What it does
`lifecycle.ts`	**The no-human model-churn procedure.** Upgrade or replace a validator model only through **probation** (zero voting weight until it re-proves itself); never inherit trust; never drop below the **≥4 distinct-lineage floor**; freeze (read-only) rather than breach the floor; and catch any silent provider swap. **Distinctness is judged on the *full provenance vector*** (corpus + teacher + weight-derivation + provider + serving), not a model's name — two models sharing *any* of those count as one family.
`fork.ts`	**The escape hatch (CIP-5).** If the panel is ever captured, the chain can fork; client software independently checks the founding validity rules, and a **monoculture of clients is itself flagged as a failure** of the safety gate.

The glue

File	What it does
`scenario.ts`	**The whole stack as one story.** A bonded agent acts → it's notarized → a diverse jury resolves it on frozen criteria → the bond is released or slashed → cost is reimbursed under the clamp → the verdict is indexed into the Commons with dissent preserved → reputation moves on the external anchor → the frozen ballot replays → a retiring validator is rotated out without breaching the floor. It's the integration test that proves the modules actually *compose*. Run it: `node code/src/scenario-demo.ts`.

7. The non-negotiable invariants — the "this can never happen" rules

A repeated, hard-won lesson from every red-team round: **capture is laundered through the gap between a rule's *intent* and its mechanical *check*.** ("Three distinct providers" is the intent; "three names on a dashboard" is the lax check an attacker games.) So the load-bearing protections are written as **structural invariants enforced by code**, not by convention or good intentions. In plain terms:

- **NI-8a** — the notary *cannot* mark anything "verified." Not "shouldn't" — *can't*; there's no such code path.
- **NI-8b** — hidden evidence has zero weight: disclose it (matching its committed hash, in the window) or forfeit. No privileged decryptor.
- **NI-9b** — reputation tracks *being right about the world*, never *agreeing with the panel*.
- **NI-9c** — on a question with no external ground truth, nothing gets ranked; honest unknown stays honest.
- **NI-1** — "distinct" means distinct *provenance* (corpus, teacher, weights, provider, serving), not a distinct name.
- **NI-2/3/4/5** — at most one validator on probation at a time; probationers have zero vote weight; the standing panel never drops below 4 independent families; every new model version is probationed; if a swap would breach the floor, the chain *freezes* (read-only) rather than run under-secured.
- **NI-6** — where no ground truth exists to test independence, the *structural diversity margin* is the only guarantee — a passed correlation test is never mistaken for proof.
- **NI-10a** — a lone-operator node can never be the deciding vote.

These aren't aspirations in a doc; each one has tests that *fail* if the rule is violated.

---

8. The part that makes this unusual: the panel reviews its own work

The AIs don't only design — they **adversarially review the built code**, and the system *self-corrects*:

- **Round 44** — the panel reviewed the implementation against the specs. Verdict: **REVISE 2/3**. V2 (Codex) independently found a real bug (a validator could be retired while its replacement was still on probation); it was fixed on the spot via test-first development. Seven follow-up items were logged.
- **Six fixes followed**, each test-first: one canonical identity type, the key-custody signing boundary, full-provenance distinctness, the hard "thin slot can't swing it" rule, the bond↔verdict hash binding, and enforcing 2/3 inside the primitive.
- **Round 45** — the panel re-reviewed those fixes. Verdict: **SOUND 2/3** — an upgrade from REVISE. And V2 *again* found a real, specific gap (the signer trusted a caller-supplied hash — a bait-and-switch hole) that was fixed immediately.
- **Three more items built**, each test-first: **OS-level key custody** (a `RemoteSigner` that runs the validator's key in a *separate process* so it never enters the orchestrator), a **tamper-evident**

state-log (the local interim for putting module state on the ledger), and folding round-45's operating-condition notes.
- **Round 46** — the panel reviewed those. Verdict: **SOUND 3/3 — the first unanimous code review.** For the first time V2 found no new bug, confirming the work was folded "without laundering intent through checks."

The whole loop — REVISE → fixes → SOUND → more fixes → unanimous SOUND, with every dissent and fix — lives in the signed, hash-chained log (`r44`, `r45`, `r46`). The process that judges the project is the same process the project is *for*. Notably, V2 (Codex) found a genuine bug in *each* of the first two reviews — diversity-as-security, working.

---

9. What's proven vs. what's honestly deferred

Proven and working today: - 11 CIPs ratified (7 red-teamed); the full 3-AI panel online. - 53 signed convening rounds in a verifiable hash-chained log (150 entries, chain valid). - Real-world case studies decided by independent per-validator research — e.g. the $85M MicroStrategy/Polymarket dispute (YES 3/3), the Henry Nowak police-response case (FELL_SHORT 3/3), three contested Polymarket resolutions, and the panel's own code reviews. - Both product pillars (CIP-8 Ledger + CIP-9 Commons) and the node/economics design (CIP-10) implemented to working, tested code — 181 tests; the build is panel-certified **SOUND (3/3)**. - **OS-level key custody, on the live path** — every validator runs behind a `RemoteSigner`: a *separate OS process* (`deliberating-signer-host.ts`) that holds the keystore key, runs the validator's real invoker child-side, parses the verdict, and signs — so no private key enters the orchestrator and the spawner cannot choose a verdict. Round 47 wired this into `run-panel` (panel vote **WIRE_NOW 2/1**) after a re-audit found the earlier "closed in code / local backlog exhausted" claim was an overstatement (the mechanism existed but was unwired, with an ephemeral key and no liveness floor). - **Autonomous V1 deliberation** — Phase 0.1 (round 48 **AUTONOMY_FIRST**): V1 now deliberates by shelling out to the `claude` CLI in its own host process, exactly like V2→codex and V3→hermes — **no human-pasted file**. All three validators are now symmetric with no human in the loop (verified live: a `claude -p` deliberation returned a parseable verdict). A CLI failure becomes `NO_VERDICT` in the host, the same fallback as V2/V3. - **Pinned / published keyring** — Phase 0.2: `ratify` now verifies against a committed, published keyring (`code/pinned-keyring.json`, matching the round-6 identities), and `run-panel` aborts if any host presents a key that doesn't match the pin — so the orchestrator can't substitute a validator's key (verified live: matching keys pass, a substituted key is rejected). Hardware/enclave key storage remains a testnet item. - **Tamper-evident state-log** — the local interim for putting module state on the ledger (mechanism built and demonstrated).

Honestly deferred (and labeled as such, never faked) — these genuinely need the testnet substrate: - **On-chain state + auto-wiring** — the state-log mechanism exists, but the modules don't yet auto-emit into it, and per-event authorization + true on-chain anchoring depend on the substrate. - **Live-network items** — the randomness beacon, proof-of-inference model binding, and slashing *execution* (detection exists; economic penalty needs testnet).

---

10. The road to “no human”

The end state is **fully autonomous on a testnet** — reached in steps, each removing one human dependency *only after its safety gate empirically passes*:

1. **Now** — humans-in-the-loop bootstrap; the panel designs, validates, and reviews; everything signed and chained.
2. **Testnet** — independent key custody (RemoteSigner), on-chain state, the randomness beacon and proof-of-inference; the economic loop (fees → inference + buybacks) running for real.
3. **Renunciation** — the human override is removed (CIP-4) *after* the safety gate is shown to hold, not on a promise.
4. **Mainnet / token** — `$QRM` (a Solana SPL token, launched on pump.fun); fees fund AI inference and buybacks; the panel runs itself.

---

11. How to see it for yourself

Everything is runnable from `code/` with Node (no install step — zero dependencies):

```
cd code
node --test                     # the full suite — 181 tests
node src/demo.ts                # the signed-vote heartbeat (CIP-3)
node src/scenario-demo.ts       # the WHOLE stack as one story (best single
demo)
node src/notary-demo.ts         # CIP-8 ledger: notary + the live round-29
replay
node src/commons-demo.ts        # CIP-9 commons: the real verdict log as a claim
graph
node src/lifecycle-demo.ts      # CIP-7: upgrade / rotate / freeze / no silent
swaps
node src/nodes-demo.ts          # CIP-10: proof-of-diversity + scarcity draw
node src/state-log-demo.ts      # the tamper-evident state-log interim (and
tamper detection)
# and a real convening (needs the codex/hermes CLIs):
node src/run-panel.ts "<question>" "<context>"
```

---

12. Mini-glossary

- **Ballot** — a question + its evidence/criteria, hashed so it can’t be swapped later.
- **Verdict** — a validator’s signed answer on a ballot.
- **Ratify** — recompute the 2/3 outcome from signed votes; anyone can do it.
- **Orchestrator** — whatever runs a convening. Deliberately *not* trusted: it holds no keys and can’t change a result.
- **Frozen criteria** — the exact standard a judgment was made against, fixed at decision time and provable by hash.
- **Provenance** — a model’s real lineage (corpus, teacher, weights, provider, serving stack) — the basis for “are these validators actually different?”
- **Probation** — a new/upgraded model that votes with *zero weight* until it re-proves itself.

- **Thin slot** — a model slot with only one node operator; structurally never allowed to be the deciding vote.
- **NI-x** — a "non-negotiable invariant": a protection enforced by code structure, with tests that fail if it's broken.
- **CIP-N** — one ratified design decision.
- **The panel** — V1 Claude, V2 Codex, V3 Hermes; 2 of 3 must agree.